PROJECT DEVELOPMENT REPORT

Real-Time Driver Fatigue and Distraction Detection System

PHASE 1: WEBCAM INITIALIZATION AND VIDEO CAPTURE

Project	Real-Time Driver Fatigue and Distraction Detection System
Phase	Phase 1 — Webcam Initialization and Video Capture
Core technologies	Python 3.11, OpenCV, NumPy
Development tools	VS Code / Antigravity, Git and GitHub
Status	Implementation and reported automated tests complete

Prepared as a development record for the project team.

1. Project Overview

Phase 1 establishes a reliable webcam-based video capture foundation for the driver monitoring application. It captures live frames, displays performance information, supports screenshot capture and prepares a modular base for later computer-vision and machine-learning components.

2. Objectives

• Initialize the system webcam successfully.

• Capture live video frames using OpenCV.

• Display video with real-time performance information.

• Calculate and display frames per second (FPS).

• Capture screenshots and save them in the screenshots directory.

• Support horizontal mirroring for a natural preview.

• Release camera resources and close windows safely.

• Keep the code modular for later integration.

3. Technology Stack

Technology	Purpose
Python 3.11	Main programming language
OpenCV	Webcam access and video processing
NumPy	Numerical and image-array operations
VS Code / Antigravity	Development environment
Git and GitHub	Version control

4. Implementation Details

4.1 Webcam Initialization

A VideoCamera class was implemented in detection/video_capture.py. It supports configurable camera indexes, requests preferred resolution and frame rate, retrieves negotiated resolution, and handles camera initialization failures. On macOS, the implementation selects the AVFoundation backend with a fallback option.

4.2 Real-Time Video Capture

The application continuously reads frames from the webcam and displays them through an OpenCV window. Unsuccessful frame reads are handled without allowing a single dropout to crash the application.

4.3 FPS Calculation

The implementation calculates instantaneous FPS and smooths the displayed value using an exponential moving average (EMA):

$$FPS_{rolling} = 0.9 \times FPS_{previous} + 0.1 \times FPS_{instantaneous}$$

4.4 Horizontal Webcam Mirroring

The preview uses cv2.flip(frame, 1). The original sensor frame remains unmirrored for future MediaPipe and machine-learning processing, while a separate mirrored frame is used for display. HUD text is drawn after mirroring so

it remains readable.

4.5 Screenshot Capture

Pressing S saves the current mirrored preview to the screenshots directory. Filenames include timestamps to avoid overwriting earlier captures.

4.6 Keyboard Controls

Key	Action
S	Capture and save a screenshot
Q	Exit the application
ESC	Exit the application

4.7 Resource Management

The camera is released during shutdown, OpenCV windows are closed, and context-manager support is available for safe camera lifecycle handling.

5. Project Architecture

```
driver-fatigue/
■■■ main.py
■■■ requirements.txt
■■■ README.md
■■■ config/
■■■ detection/
■   ■■■ video_capture.py
■■■ features/
■■■ models/
■■■ decision_engine/
■■■ alerts/
■■■ visualization/
■■■ dashboard/
■■■ dataset/
■■■ tests/
■■■ screenshots/
■■■ docs/
    ■■■ architecture.md
```

The project uses a modular structure. Phase 1 imports only the video-capture functionality and its required dependencies; future modules are kept isolated to avoid unnecessary eager imports.

6. Development Environment

The project environment uses Python 3.11 in a dedicated virtual environment. Dependencies are installed phase-wise so packages required only by later phases do not block initial webcam development.

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install opencv-python numpy
```

The active interpreter should point to the project .venv directory. This prevents accidental installation into the system Python environment.

7. Testing and Verification

The reported automated test command is:

```
python -m unittest discover -s tests -v
```

The implementation report records 18 automated tests passing, including camera initialization, configuration, frame reading, FPS updates, screenshot saving, context-manager cleanup and horizontal mirroring. It also reports that heavy future-phase dependencies are not imported by the Phase 1 video module.

Test area	Reported result
Camera initialization and configuration	Passed
Frame reading and failure handling	Passed
FPS calculation	Passed
Screenshot saving	Passed
Context manager and cleanup	Passed
Horizontal mirroring	Passed
Dependency isolation	Passed

Note: Automated unit tests validate the tested code paths. Physical webcam behavior should also be confirmed manually on the target machine.

8. Challenges and Solutions

Challenge	Resolution
Python version mismatch	Use a dedicated Python 3.11 virtual environment.
OpenCV import error	Install OpenCV in the active environment.
Unnecessary eager imports	Keep package __init__.py files lightweight.
Pygame SDL compilation issue	Defer Pygame until the audio-alert phase.
Unmirrored webcam preview	Create a separate horizontally flipped display frame.
Screenshot orientation	Save the mirrored display frame.

9. Future Development Plan

Phase	Planned work
Phase 1	Webcam initialization and video capture — completed
Phase 2	MediaPipe facial landmark detection
Phase 3	Eye, mouth and head-pose feature extraction
Phase 4	Hybrid machine-learning classification
Phase 5	Audio alert integration
Phase 6	Streamlit dashboard development
Phase 7	Testing, optimization and final integration

10. Conclusion

Phase 1 establishes the webcam and video-capture foundation for the Real-Time Driver Fatigue and Distraction Detection System. The implementation includes live video streaming, FPS monitoring, screenshot capture, mirrored preview, and safe camera cleanup. The modular architecture is prepared for subsequent facial landmark detection and driver-state analysis.

Next milestone: Phase 2 — MediaPipe facial landmark detection, including face, eye and mouth landmarks.